

LangChain & LangGraph

LangChain과 LangGraph

02 LangGraph

목차 02 LangGraph

1. Naïve LLM
2. Agent를 활용한 논문 작성
3. LangGraph의 3가지 구성 요소

1. Naïve LLM



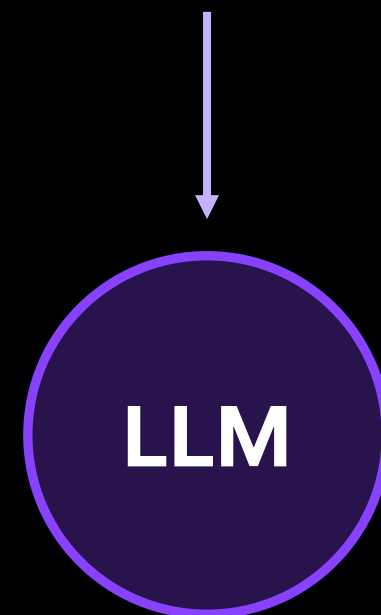
Input text



**Desired
output**



Input text



Desired
output

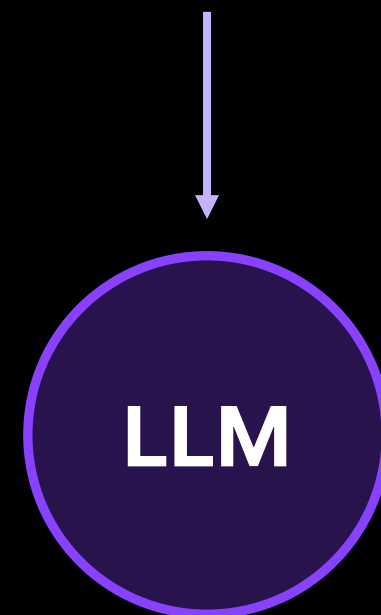
Naïve LLM

[구조]

- Input text를 받아서 처리함
- Prompt execution 등 간단한 function을 통해 output을 생성함



Input text



Desired
output

Naïve LLM

[구조]

- Input text를 받아서 처리함
- Prompt execution 등 간단한 function을 통해 output을 생성함

[한계점]

- 구조가 간단하기 때문에, 여러 단계가 필요한 task를 처리하기 어려움

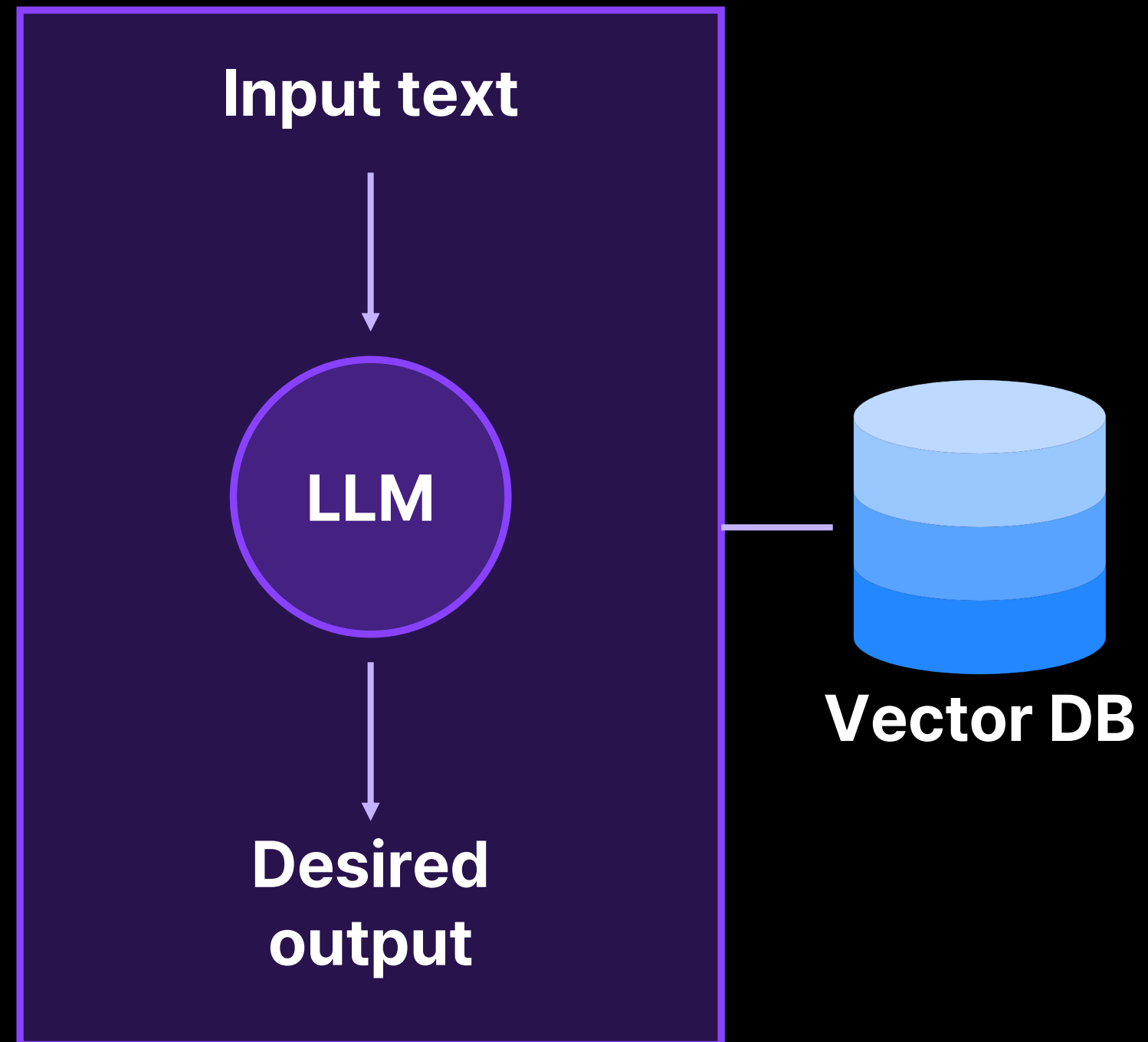


[RAG]



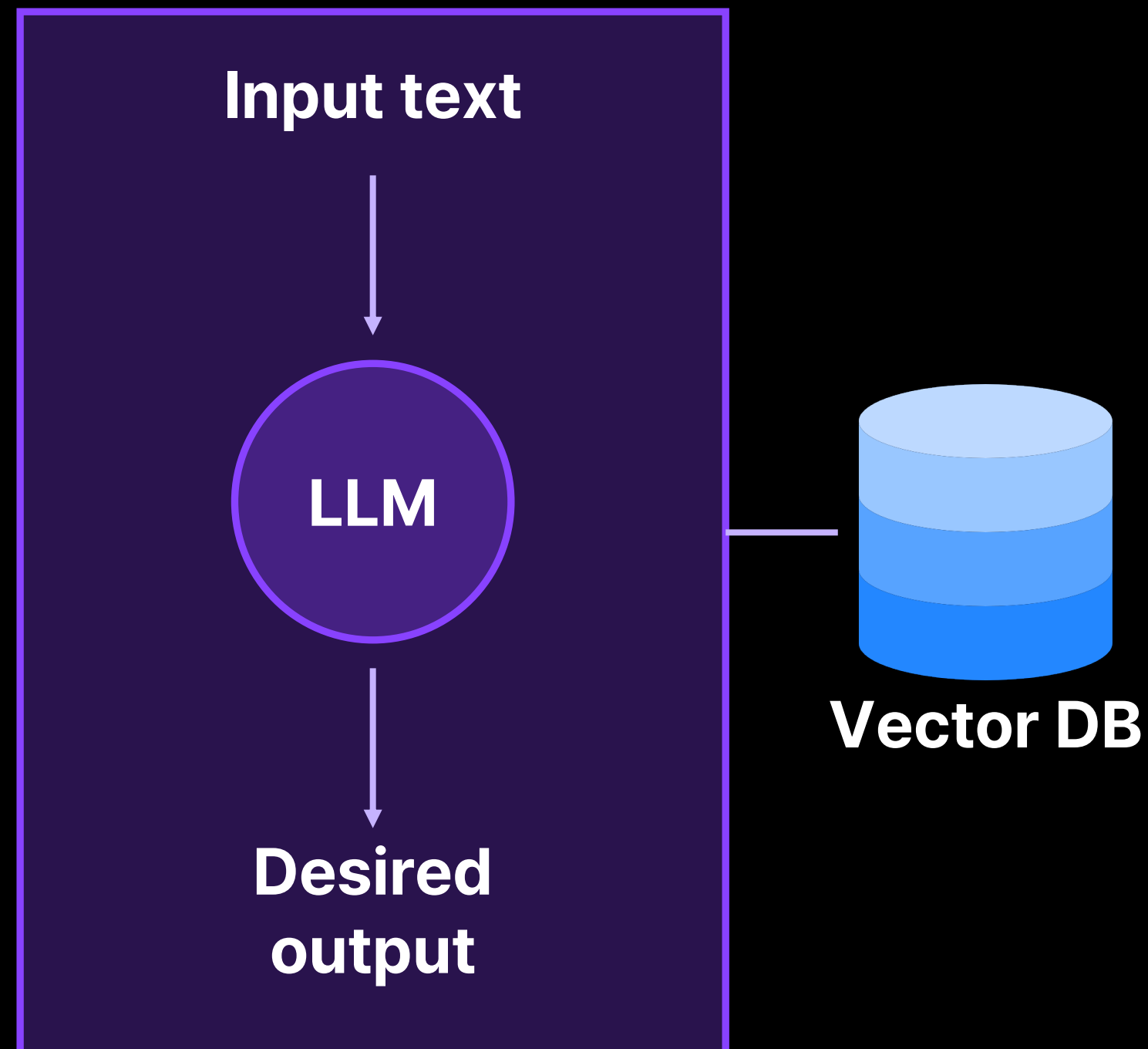


[RAG]





[RAG]



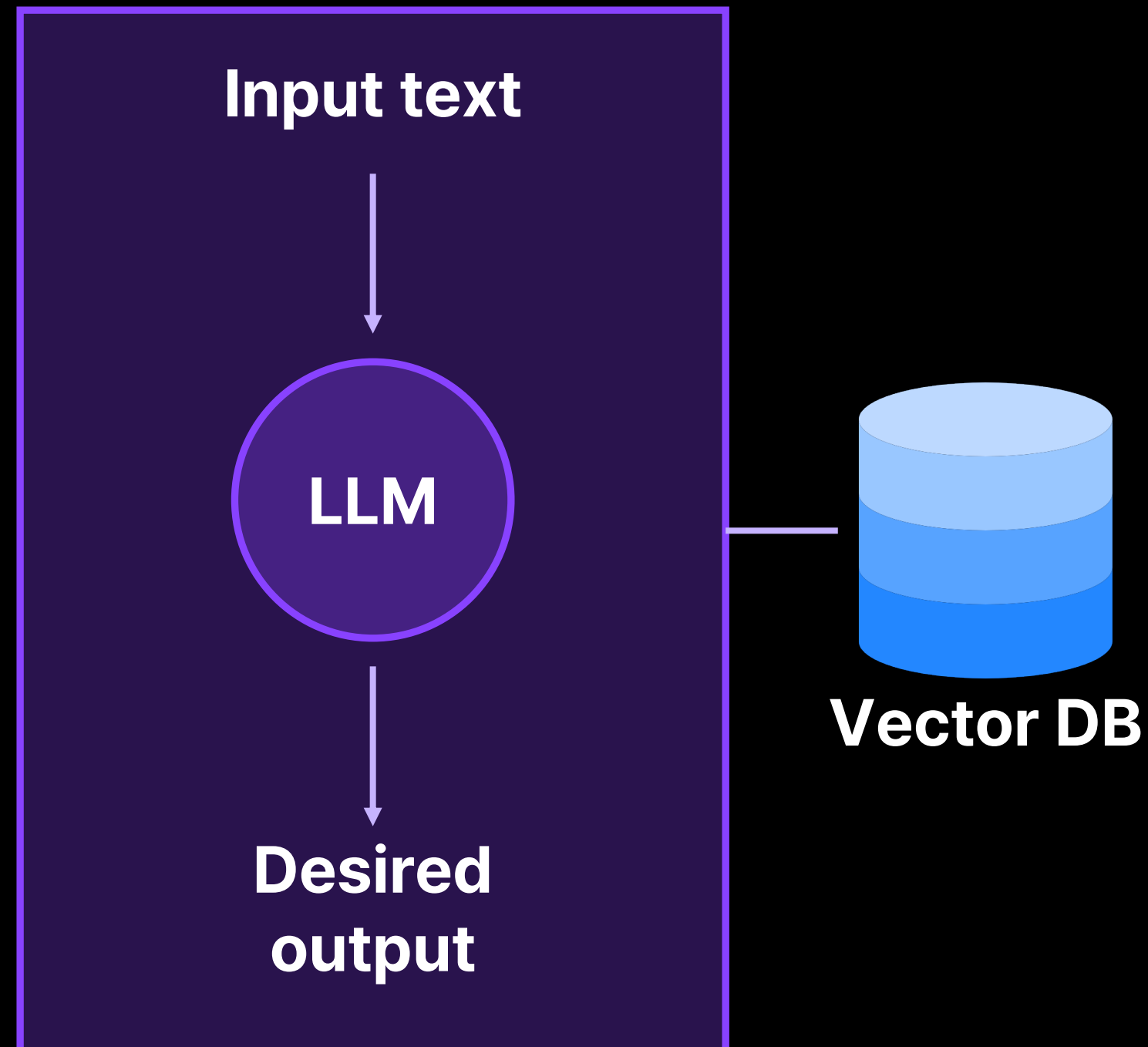
Naïve LLM with RAG

[구조]

- Input text를 받아서 처리함
- RAG를 통해 input text와 관련한 정보를 추가함



[RAG]



Naïve LLM with RAG

[구조]

- Input text를 받아서 처리함
- RAG를 통해 input text와 관련한 정보를 추가함

[한계점]

- Input text를 분석하여 context를 이해할 수 있어야 보다 고도화된 task를 수행할 수 있음
- Tool 사용 불가



LLM을 활용한 정형화된 아웃풋을 연결함으로써,
복잡한 문제를 해결할 수 있는 유연한 cognitive 아키텍처를 구축

2. Agent를 활용한 논문 작성



Agent를 활용한 논문 작성 (예시)

LangChain과
LangGraph

02 LangGraph

2. Agent를 활용한 논문
작성



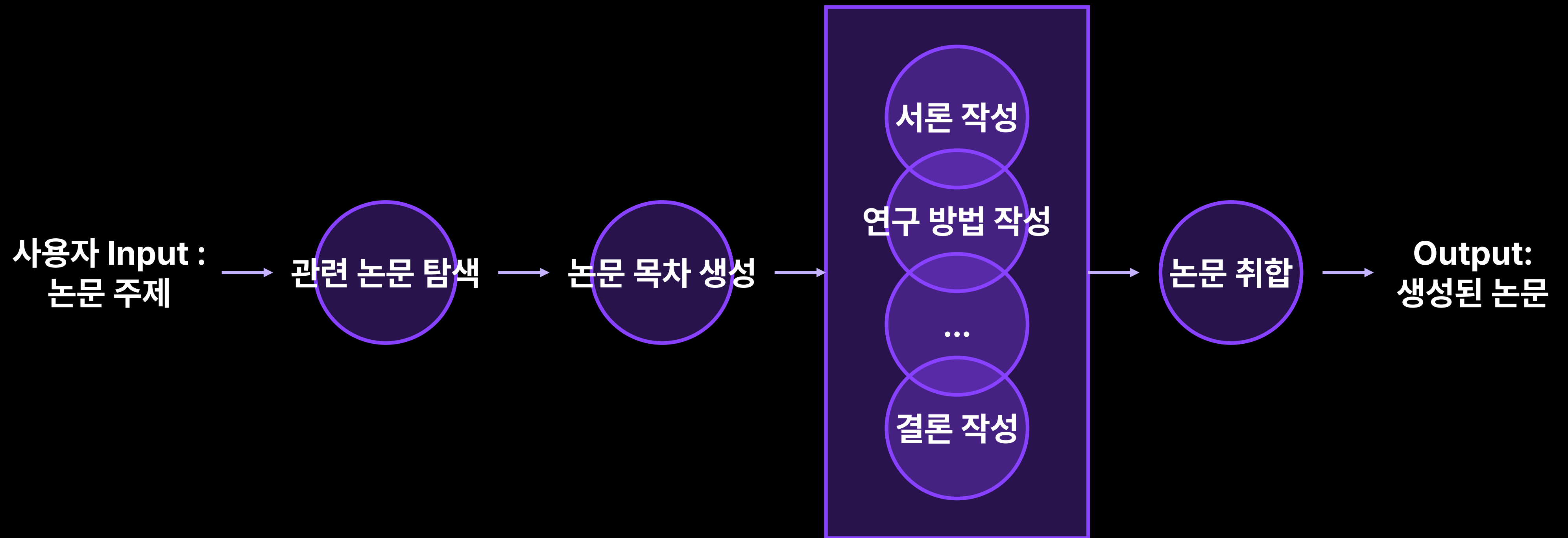


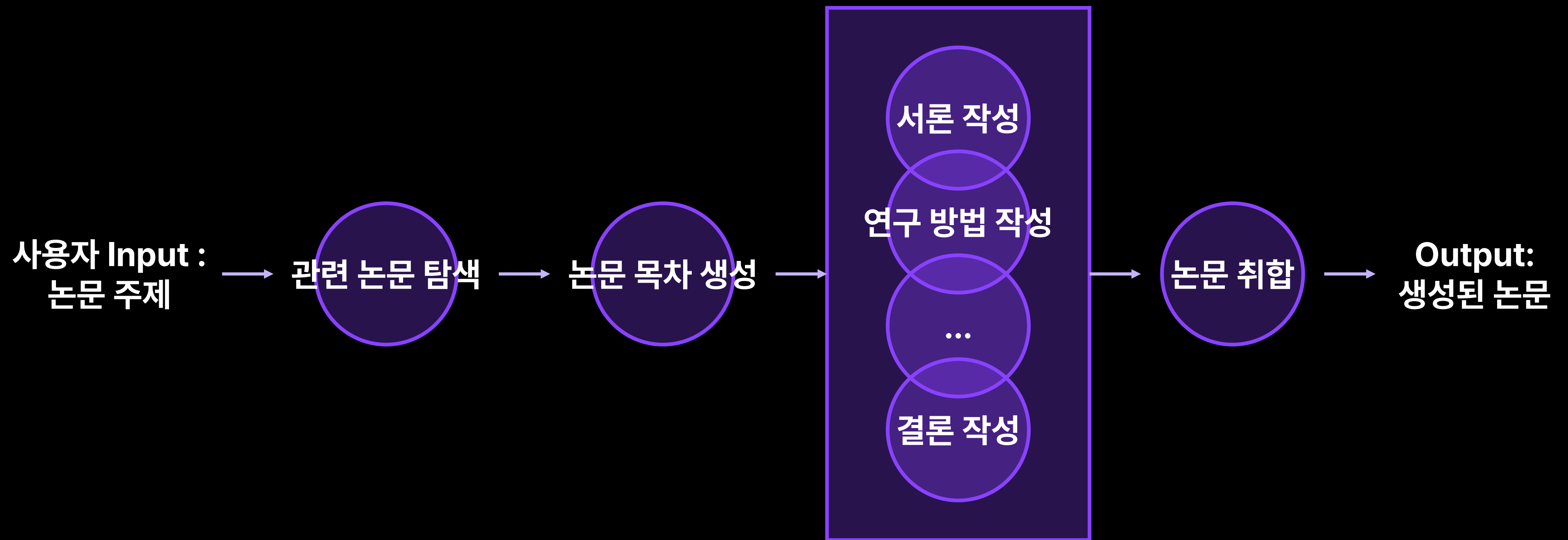
Agent를 활용한 논문 작성 (예시)

LangChain과
LangGraph

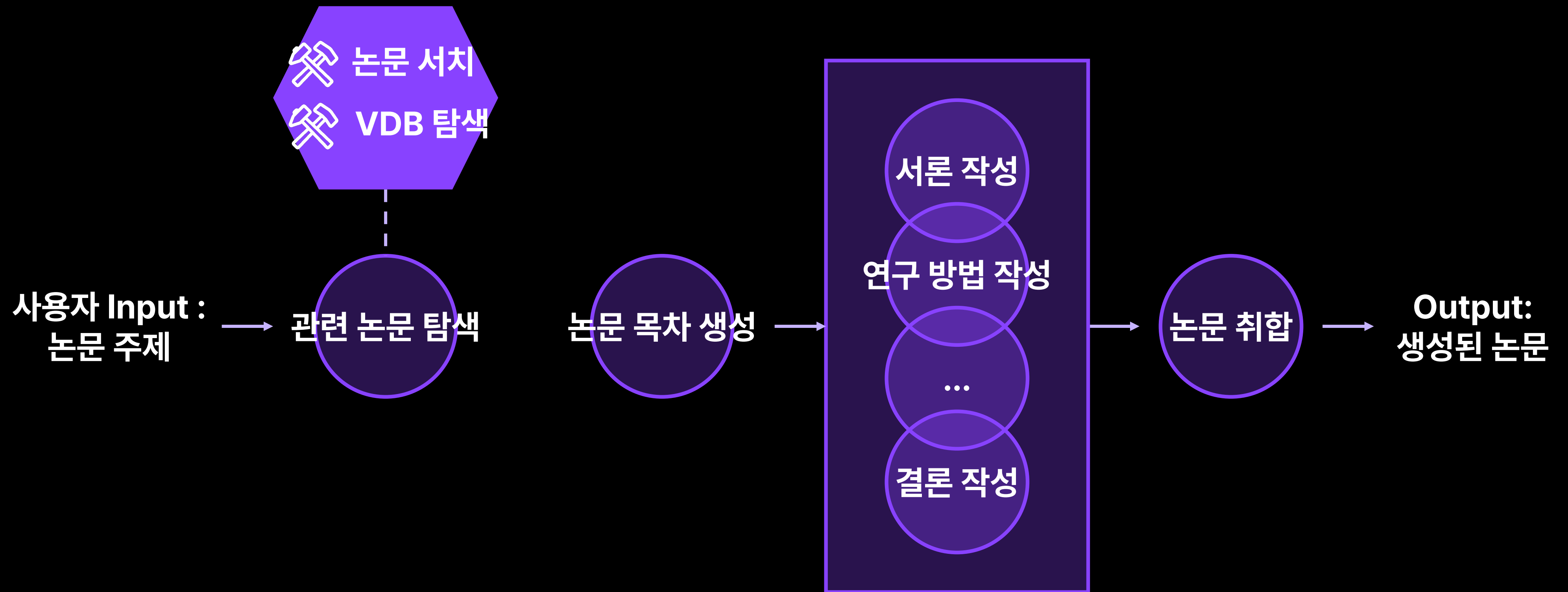
02 LangGraph

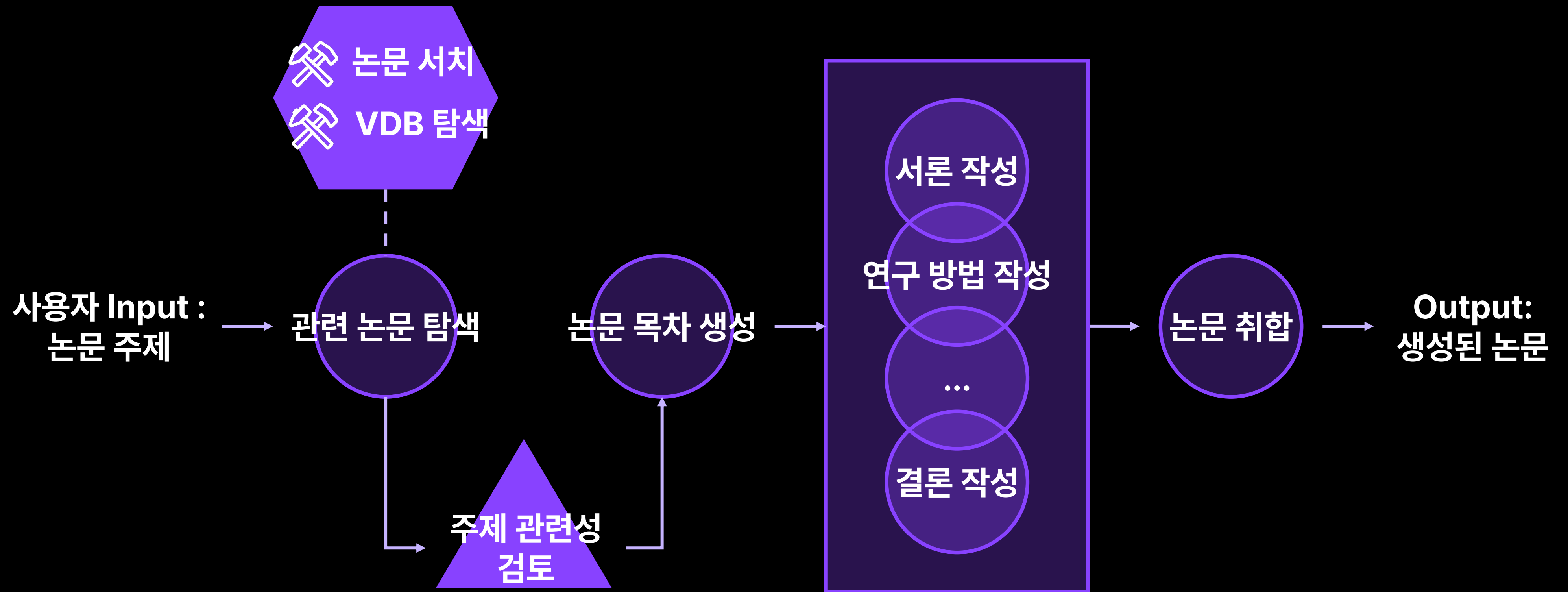
2. Agent를 활용한 논문
작성

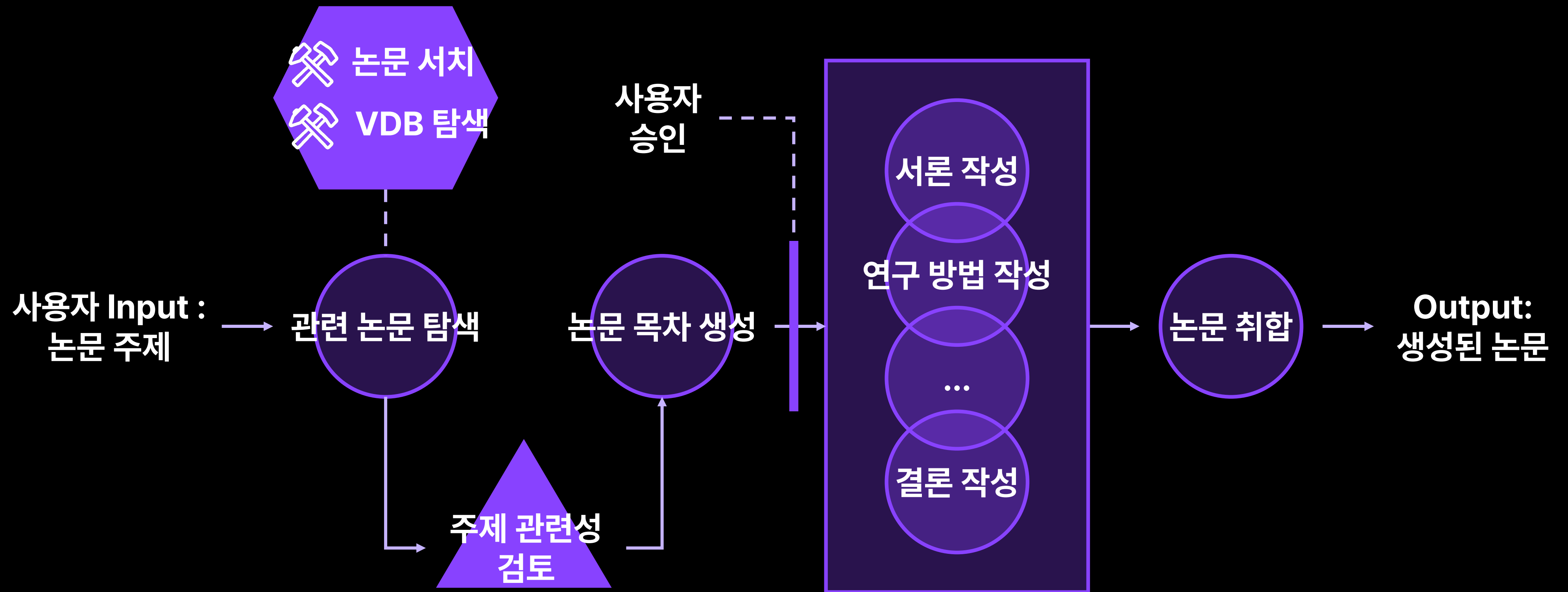


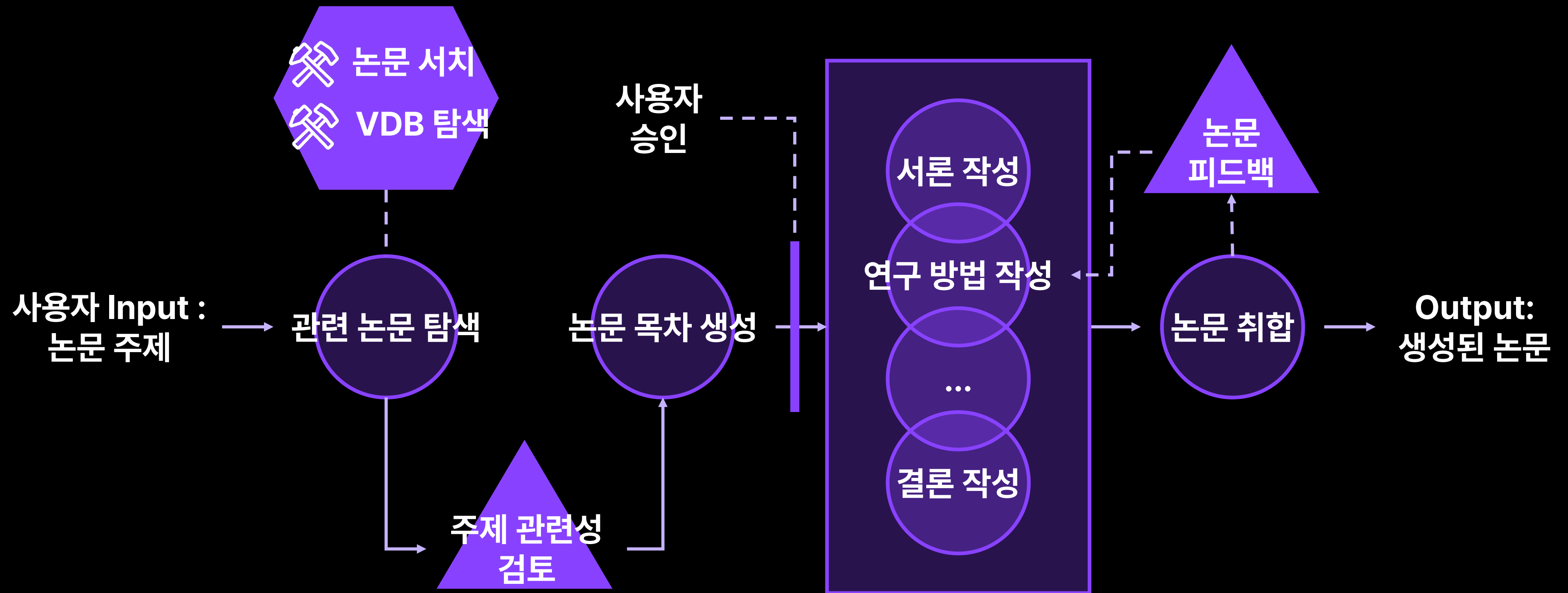


유연한 아키텍처 구성을 활용하여 복잡한 task를 breakdown
각 단계마다 정형화된 output을 제공함으로써 안정화된 시스템 설계









피드백, 사용자 인풋, 외부 툴 사용 등을 통해 90점에서 100점짜리 agent 구축 가능



LangGraph를 사용하는 이유

LangChain과
LangGraph

02 LangGraph

2. Agent를 활용한 논문
작성

Cycles and
branching

다양한 스텝들을 유기적으로 연결
하기도 하며, 동시 다발적으로 실행하는 등 유연한 아키텍처 구현



Cycles and
branching

다양한 스텝들을 유기적으로 연결
하기도 하며, 동시 다발적으로 실행하는 등 유연한 아키텍처 구현

LangChain
연동

LangChain과 Python의
모든 기능 활용 가능



Cycles and
branching

다양한 스텝들을 유기적으로 연결
하기도 하며, 동시 다발적으로 실행하는 등 유연한 아키텍처 구현

아키텍처의
지속성

각 스텝의 아웃풋을 중단하고 저장하던지, 이전 상태로 돌아가는 등 아키텍처가 유지되는 상태에서 자유롭게 '시간' 조정 가능

LangChain
연동

LangChain과 Python의
모든 기능 활용 가능



Cycles and
branching

다양한 스텝들을 유기적으로 연결
하기도 하며, 동시 다발적으로 실행하는 등 유연한 아키텍처 구현

아키텍처의
지속성

각 스텝의 아웃풋을 중단하고 저장하던지, 이전 상태로 돌아가는 등 아키텍처가 유지되는 상태에서 자유롭게 '시간' 조정 가능

LangChain
연동

LangChain과 Python의 모든 기능 활용 가능

사용자
상호작용

사용자들의 개입이나 인풋이 필요한 경우 실행을 중단하고 인풋을 요구 하는 등 human-in-the-loop이 가능함

3. LangGraph의 3가지 구성 요소



LLM을 활용한 정형화된 아웃풋을 연결함으로써,
복잡한 문제를 해결할 수 있는 유연한 cognitive 아키텍처를 구축



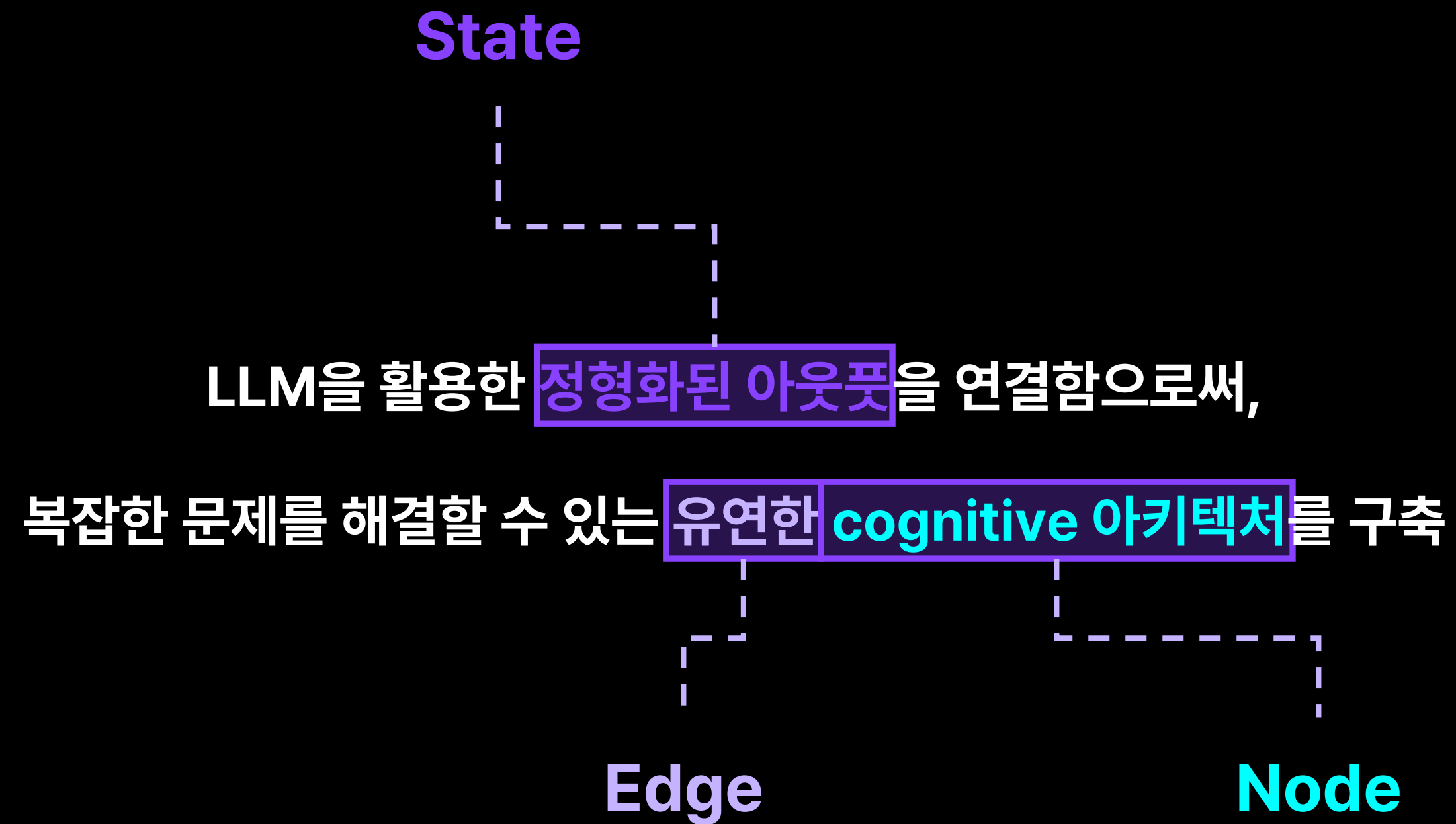
LLM을 활용한 정형화된 아웃풋을 연결함으로써,
복잡한 문제를 해결할 수 있는 유연한 **cognitive 아키텍처**를 구축

Node



LLM을 활용한 정형화된 아웃풋을 연결함으로써,
복잡한 문제를 해결할 수 있는 **유연한 cognitive 아키텍처**를 구축

Edge Node





State

- LangGraph 전체에 걸쳐
사용될 데이터
- 정형화된 input과 output
- 각 노드를 통과하면서
업데이트



State

- LangGraph 전체에 걸쳐 사용될 데이터
- 정형화된 input과 output
- 각 노드를 통과하면서 업데이트

Node

- 실제 작업을 수행
- Agent의 "논리"를 담당
- State의 데이터를 input으로 받고, 업데이트 후 output으로 제공



State

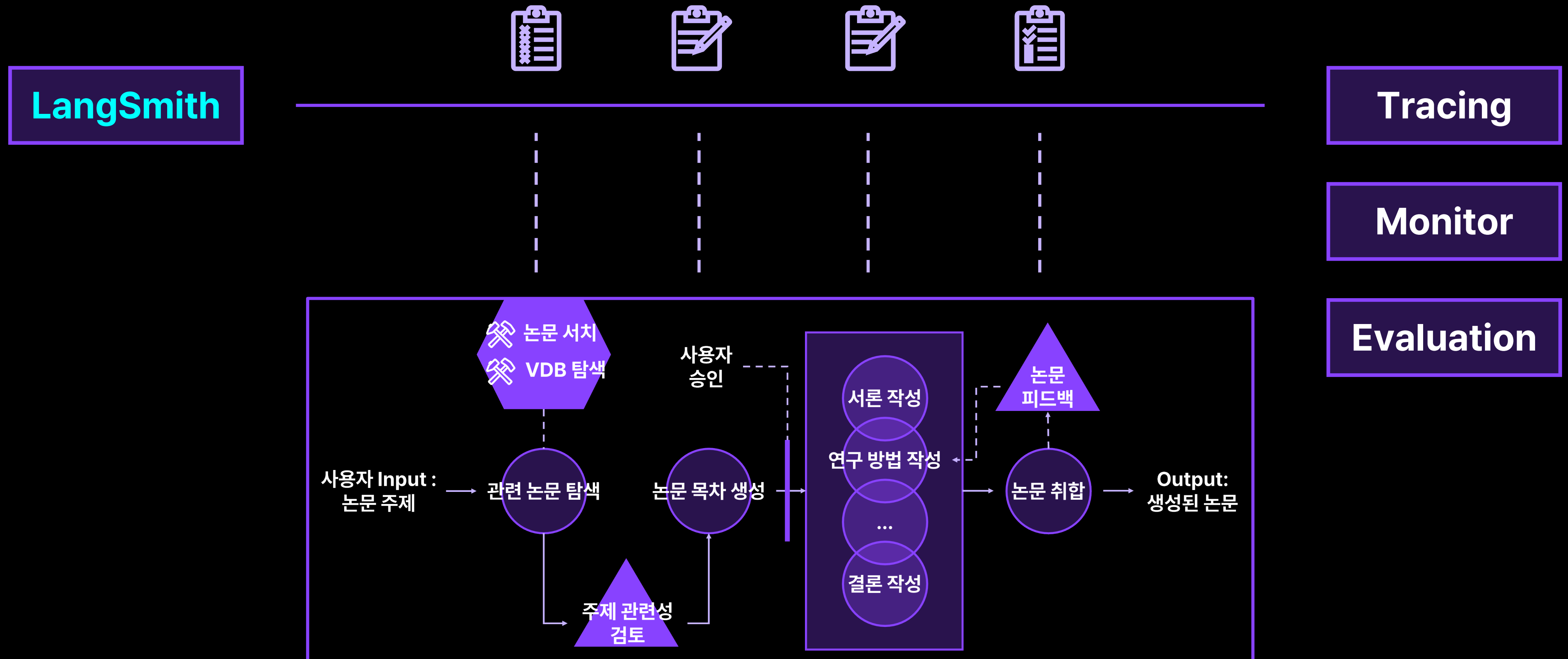
- LangGraph 전체에 걸쳐 사용될 데이터
- 정형화된 input과 output
- 각 노드를 통과하면서 업데이트

Node

- 실제 작업을 수행
- Agent의 "논리"를 담당
- State의 데이터를 input으로 받고, 업데이트 후 output으로 제공

Edge

- 각 노드들이 상호작용 하는 방식 정의
- 조건이나 일방적 전달, 사용자 input 등 다양한 형태의 상호작용 설정 가능
- 계층 구조, linear, 반복, 동시 실행 등 agent의 "사고 방식"을 정의



LangSmith : Agent의 각 단계별 인풋과 아웃풋, 그리고 내부에 발생하는 log를 트래킹